Available online at www.sciencedirect.com
SCIENCE DIRECT®
The Journal of Systems and Software 79 (2006) 415–426
The Journal of Systems and Software
www.elsevier.com/locate/jss

Investigating the impact of usability on software architecture through scenarios: A case study on Web systems

Tamer Rafla *, Pierre N. Robillard [1], Michel Desmarais [2]

Software Engineering Research Lab, Ecole Polytechnique de Montreal, P.O. Box 6079, Station Centre-Ville, Montreal, QC, Canada H3C 3A7

Received 14 February 2005; received in revised form 19 October 2005; accepted 23 October 2005
Available online 7 December 2005

Abstract

Usability has primarily been served by separating the user interface from the remainder of the application. However, several researchers have recently determined that there is a direct relationship between architectural decisions and usability requirements. This leads us to conclude that more attention should be devoted to usability-driven architectural analysis methods. We present a case study, which involves adapting an existing software architecture analysis method (SAAM) for the purpose of deriving the interdependencies between architectural characteristics and usability requirements. More specifically, we investigate the impact on the architecture of implementing usability requirement changes. Potential design solutions that accommodate the corresponding usability mechanisms into the Web software architecture are presented, along with the rationale for applying them and the process by which they are obtained. We conclude by recommending how usability issues can be dealt with proactively during the design of the architecture, and explain the need to integrate those usability requirements into a software engineering process.
© 2005 Elsevier Inc. All rights reserved.

Keywords: Software architecture; Usability; Scenarios; Web applications; SAAM

1. Introduction

Humans play an active and essential role in the operation of interactive software, and the user interface (UI) has become an essential part of many software systems (Nielsen, 1993; Rafla et al., 2004). The architectural answer to building a UI is based on the principles of modularity and separation (Hix and Hartson, 1993; Schneiderman, 1998; Nielsen, 2000). This separation has made it easier to make modifications to the UI, and usability seemed to be primarily a property of the presentation of information. Indeed, UI had never been considered in software architecture design, due to the widespread assumption among software engineers that it has to do only with the visible part of the system. However, this view is now being challenged (Bass and John, 2003; Bevan, 1995; Landauer, 1995). A more recent perspective on the UI is that the software architecture needs to be explicitly designed so that the final system can satisfy usability requirements. Even when the presentation and functionality of a system are well designed, its usability could be greatly compromised if the underlying architecture does not support usability concerns. Moreover, it often proves difficult to make the changes to a system that are necessary to improve its usability. One reason for this is that many of the required usability-driven changes involve changes to the system that cannot be easily accommodated by the software architecture (Folmer and Bosch, 2002).

We investigate the link between usability requirements and software architecture in a case study of a Web-based application. The purpose of this paper is twofold: (1) to

* Pre-publication corresponding author. Tel.: +1 514 726 6497; fax: +1 514 340 3240.
E-mail addresses: tamer.rafla@polymtl.ca (T. Rafla), pierre-n.robillard@polymtl.ca (P.N. Robillard), michel.desmarais@polymtl.ca (M. Desmarais).
[1] Post-publication corresponding co-author. Tel.: +1 514 340 4238; fax: +1 514 340 3240.
[2] Post-publication corresponding co-author. Tel.: +1 514 340 3914; fax: +1 514 340 3240.

0164-1212/$ - see front matter © 2005 Elsevier Inc. All rights reserved.
doi:10.1016/j.jss.2005.10.014

adapt a scenario-based approach to analyzing a software architecture for usability; (2) to determine how usability requirements can be supported and study their impact on the architecture by proposing Web-specific design solutions.

We begin by discussing the scenario-based method that was used to analyze the Web architecture for usability. Web-specific design solutions are then proposed to determine how usability requirements are supported. Their impact on the architecture is investigated and the results of the late implementation of usability on the architecture are analyzed. We close by discussing how usability issues can be dealt with proactively during the design of the architecture and provide topics of future work.

2. SAAM: Software architecture analysis method

The Software Architecture Analysis Method (SAAM) (Kazman et al., 1994; Kazman et al., 1996; Clements et al., 2001) was among the first to address the assessment of software architectures using scenarios. SAAM has traditionally been applied to the development of related software qualities such as maintainability (Haggander et al., 1999; Bengtsson and Bosh, 2002; Dobrica and Niemela, 2002). Since very few architectural assessment techniques for usability exist, can SAAM also be considered for usability? We explore this avenue after briefly describing the method. SAAM (Fig. 1) consists of the following four steps:

- Scenario development: The first step is to illustrate the kinds of activities the system must support and the kinds of anticipated changes that will be made to it. Since usability is often defined in a very abstract fashion and is not well interpreted by software designers, scenarios are a good way of synthesizing individual interpretations of software quality into a common view, which can make usability requirements more specific.

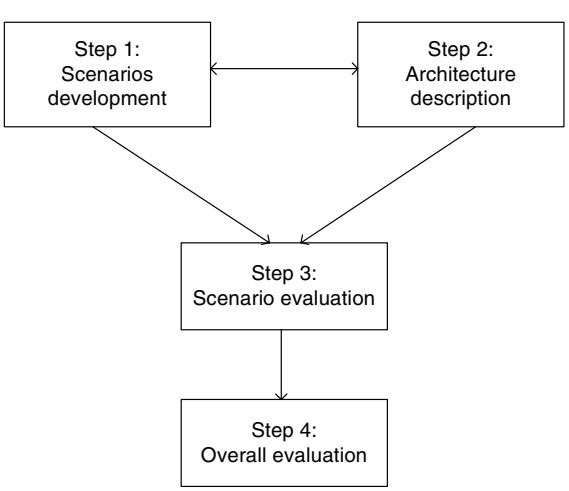

Fig. 1. Steps of the SAAM (adapted from Kazman et al., 1994).

- Architecture description: If there is no description of the system architecture, then the product is thoroughly studied and an architectural diagram is developed. This activity is to be carried out in parallel with the previous activity in an iterative mode. The final version of the architecture description, together with the scenarios, serve as the input for the subsequent activities of the method.
- Scenario evaluation: SAAM evaluates a scenario by investigating which architectural elements are affected by that scenario. The impact of the modifications associated with each scenario is estimated by listing the components that are affected.
- Overall evaluation: Finally, following scenario evaluation, the results are interpreted to draw conclusions concerning the software architecture and to assess the impact of the usability scenarios on the architecture.

In the sections below, the usability-driven analysis of the architecture is conducted by elucidating the four SAAM phases.

3. Step 1: Usability scenarios development

In this section, step 1 of the method is performed. Bass and his team at the Software Engineering Institute identified a collection of architecturally significant scenarios, which expressed a usability issue (Bass et al., 2001). Those scenarios are common to many interactive systems and are not related to the domain functionality of any system. The ones that will inevitably increase the usability of the system serving as the case study for this work are extracted, and their respective usability benefits are illustrated (Table 1). These benefits encompass the following: indicators of efficiency, problem-solving and learnability (effectiveness) and customer satisfaction (Constantine and Lockwood, 1999; Shackel, 1991; ISO 9241, 1998). These scenarios are applicable to Web applications and can be considered for other Web systems suffering from the same usability issues identified here. However, we do not claim that this set of usability scenarios covers all the usability problems identified in Internet applications.

- Checking for correctness: Users may make errors they do not notice. Depending on the context, error correction should be enforced directly or suggested through system prompts. Mistakes made entering credit card billing information, for example, can require costly manual follow-up. The checking-for-correctness scenario will be validated if and only if a mechanism allowing for all entered fields to be validated is incorporated.
- Retrieving forgotten passwords: Users may forget their passwords, and retrieving them may be difficult or may cause lapses in security. Systems should provide alternative strategies to grant the user access. It becomes tedious to contact the site administrator every time a password is forgotten. Since this is not always accom-

Table 1
Linking usability aspects to scenarios (adapted from Bass and John, 2003)

Scenarios	Usability aspects						
	Effectiveness				Efficiency		Satisfaction
	Accelerates error-free portion	Reduces impact of slips	Prevents mistakes	Accommodates mistakes	Supports problem-solving	Facilitates learning	Increases confidence and comfort
Providing good help					✓	✓	
Modifying interfaces	✓			✓			
Checking for correctness		✓	✓	✓	✓		
Retrieving forgotten passwords				✓			
Supporting International use	✓	✓	✓	✓	✓	✓	✓

plished in a timely fashion, a user should be able to retrieve it without help. Thus, retrieving forgotten passwords is another scenario worth considering.

- Modifying interfaces: Poorly designed interfaces increase user errors. Looking at the system from a user's point of view reveals that several important aesthetic issues that need to be considered to make the site more usable, are not addressed. Good hypermedia design practices should be used to develop applications that are easy to use, provide friendly navigational spaces and seamlessly integrate the underlying transactional behavior.
- Providing good help: It does not matter how intuitive an application is; If it is rich in functionality, then the user will inevitably need some kind of assistance. Moreover, as Web applications become more complex, the importance of providing help grows inexorably. Since the system is not equipped with a help system, and considering that some fields are difficult to comprehend, providing good help is another scenario that should be taken into account.
- Supporting international use: The increasing diversity of the Internet has created a tremendous number of multilingual resources on the Web. Users may want to configure an application to communicate in their own language or according to their cultural norms. Systems should be easily configurable for deployment in multiple languages.

4. Step 2: Architecture description

4.1. Replan: The meeting management system

Implemented using the Unified Process for EDUcation (UPEDU) (Robillard et al., 2003), Replan is a Web-based meeting management system aimed at the organizers of meetings where the numbers of participants and their geographic distribution make scheduling difficult. This system would allow meeting coordinators to send availability requests to a set of individuals, so that each of them can specify personal availability periods. The set of availability periods would then be graphically represented using a calendar tool to enable a coordinator to visualize the relevant information at a glance, making the scheduling decisions easier.

4.2. Replan's architecture

The Java Enterprise Edition (J2EE) Web architecture (Inderjeet et al., 2002) consists of Web browsers, a Web server and a network connection (Figs. 2 and 3). Browsers request Web pages from the Web server, which distributes JavaServer Pages (JSP) of formatted information to clients. This request is made over a network connection and uses the HTTP protocol.

In this implementation of the architecture, the JSP files not only contain the HTML code necessary to produce the UI, but are also responsible for processing the user's requests in order to perform the necessary business logic. Hence, the JSP contains a significant amount of embedded Java code responsible for storing the session and linking JSP pages together. The JSP component is in charge of request processing and the creation of any objects used by the JSP, as well as deciding, depending on the user's actions, to which JSP page the request is to be forwarded. Functionalities of this component include the validation of request parameters with JavaScript, the authentication of the user and the posting of user forms by connecting to the database through the Java component, before dispatching the request to another JSP for presentation of the response. The Java component of the system interacts with an Oracle database. The Tomcat Web server combines the components according to configuration information specifying the connection between these components. In fact, interdependent JSPs have been grouped into modules since the system is modeled from a high-level point of view and the relationship between those JSPs is not of interest to us. The application control is decentralized, because the current JSP page being displayed determines the next page to display. The page flows are embedded on the links between the pages.

We wish to verify whether or not the incorporation of usability requirements would be possible with different implementations of the Web architecture. Two architectures were selected for the following reasons:

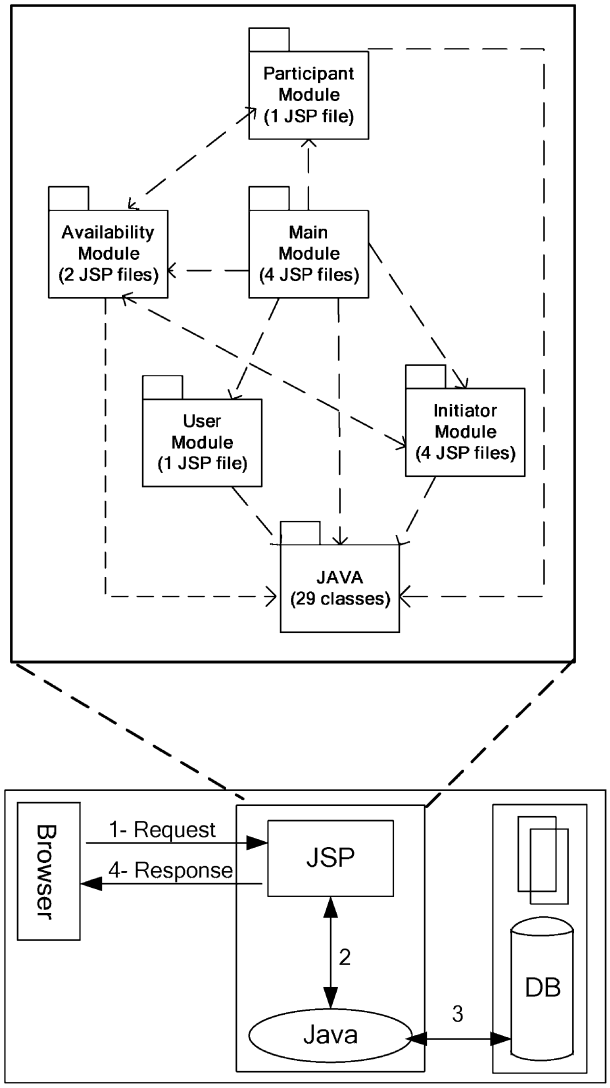

Fig. 2. Team A's architecture.

- To ensure the validity and correctness of the results.
- To show that the proposed solutions are possible with different implementations of the architecture.
- The other excluded projects suffered major drawbacks, such as failure to meet the requirements. Some features were nonexistent and the system was not fully operational.

Table 2 compares the two selected architectures, comparing the number of Java and JSP files since they are the essence of the J2EE Web architecture. The number of tables created in the database is also illustrated. Team A has 29 Java classes, accounting for 16 KB, and 15 JSP files, accounting for 50 KB. The business logic represents 76% of the overall structure of the system, while the interface represents only 24%. This first system was properly implemented, since business logic is well encapsulated in the Java classes and the JSP files were used only to embed some HTML code for the UI. However, Team B has more JSP files than Java classes. It has 24 JSP files for a total size of 200 KB, but only 7 Java classes for a total size of 74 KB. The business logic represents only 26% of the overall structure of the system, while the view represents 74%. Some business logic is incorporated in the JSP component rather than in the Java component, and the guidelines for Web architecture design were not properly followed. Figs. 2 and 3 illustrate the architecture of the two selected systems.

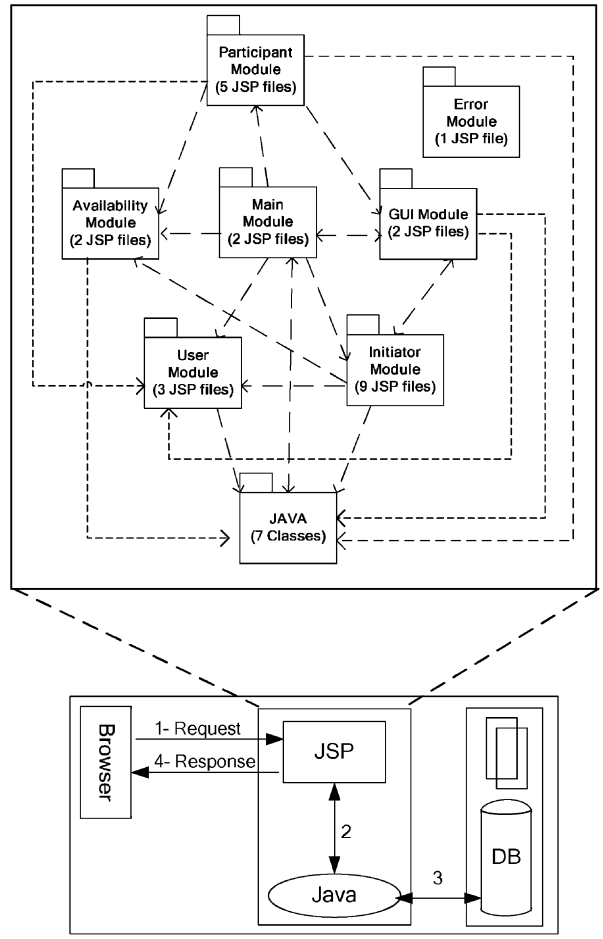

Fig. 3. Team B's architecture.

Table 2
Structure of two versions of Replan

	Java		JSP		Database
	Number	Size (KB)	Number	Size (KB)	
Team A	29	160	15	50	5
Team B	7	74	24	200	11

5. Step 3: Scenario evaluation

In this section, step 3 is exemplified. Design solutions to satisfy the chosen usability scenarios are presented. The magnitude of the impact is represented as follows:

- *No impact:* A small number of Web components are modified for the two selected architectures and the development effort is low. The required changes are categorized as architecturally independent and can still be implemented once the system has been designed.
- *Minor impact:* The number of modified architectural components is different; however, the scenarios necessitate more or less the same implementation effort. Nevertheless, it is beneficial to consider those usability requirements at earlier stages of development.
- *Major impact:* Sizeable components are reworked, and the required changes necessitate a substantial development effort. This effort is entirely dependent on the implementation of the architecture. Such modifications should be considered at the design stage due to the considerable effort they entail.

5.1. Checking for correctness

With server-side validation, the processing of the form fields is performed after they are submitted to the server. Using JavaBeans (Inderjeet et al., 2002), the designer needs to create a "validation bean" that will perform secure server-side validation on the entered data. Implementing a validation bean requires several steps. First, a data bean that has fields identical to the form fields must be designed. Second, the validation rules for these fields must be devised. These two steps are implemented in the Java component of the application. For example, if the validation of the user's form is required, then a validation bean with its respective validation rules needs to be added to the *User* class. Finally, all JSP files must be redesigned to work effectively with every validation bean that was introduced. They are slightly modified to incorporate the few lines of code responsible for initializing the bean and directing the request to the Java classes for validation. This approach separates the business logic (Java component) from the presentation (JSP component).

This approach of providing form validation entails the modification of the same number of components and files for both architectures (Figs. 4 and 5). Surprisingly, after detailed analysis of the two architectures, it became evident that the JSP contains some embedded Java code, which is responsible for storing all objects and also for transferring control to another JSP in charge of validation. Unfortunately, business logic should be encapsulated in the Java component, while the presentation resides in the JSP component. This means that all the inserted Java code in the JSP file should be moved to the Java component and the JSP file should contain only the HTML code responsible for displaying the form. However, this method involves a significant development effort in both architectures, since a large number of components are reworked. The impact of this scenario on the Web architecture is classified as intermediate.

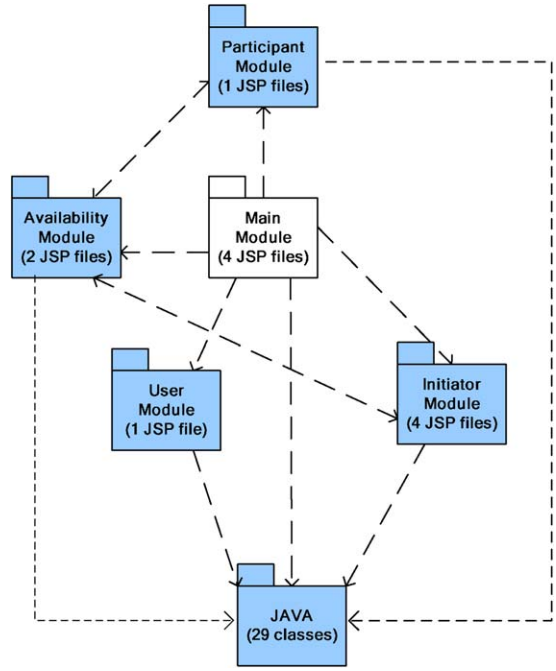

Fig. 4. Team A: server-side validation.

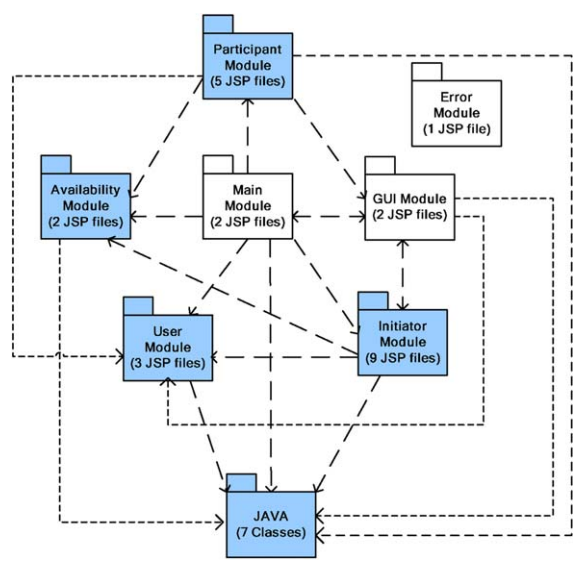

Fig. 5. Team B: server-side validation.

5.2. Retrieving forgotten passwords

The Web has moved beyond purely open content available to all. It is now commonly used to provide information that ought to be restricted in some way to members. One common method of restricting access is to ask users to enter a name ("username") and a password. This simple combination can be a source of annoyance and frustration to users because the user's password can be forgotten. Adding a link in the "login" page that will redirect the user to a "lost password" page where the user can enter his/her username is the simplest solution. Once the request is

submitted, the password is emailed to the user. This procedure is fairly simple to implement since the username is basically the email of the user according to the specifications of the system, and the modifications are reasonably straightforward to implement (Fig. 6). A few lines of Java code were added to the User class (Java component) to select the user's password from the Database. Also, a *Lost-Password.JSP* page, which is linked to the *Login.JSP* page, is created to email that password to the user. That newly created JSP is included in the Main module with the *Login.JSP*. The changes required to instigate this scenario require the developer to rework the JSP and the Java components (Fig. 6). Fortunately, these adjustments seem to be rather simple to make, since they do not require a significant programming effort and can be added at any stage of development. This solution works for both architectures. The Java component has been slightly modified for both teams and a new interface has been added to the Main Module in the J2EE component. The JSP architecture can easily accommodate the mechanism for retrieving forgotten passwords. This scenario does not have an impact on the system's architecture, since it requires a low development effort for both teams and is independent of the overall implementation of the system.

5.3. Supporting international use

In today's competitive world, many businesses are becoming global in order to reach the widest possible targeted audience. This is a problem, because Web applications developed by these organizations must be suitable for by customers who use and understand different languages. Therefore, there is an increasing need to develop and design cost-effective Web solutions capable of delivering content in multiple languages. To be considered "multilingual", an application has to support the following elemental categories of features:

- *Data representation:* supporting all the character set encodings that an end-user may use.
- *Data display:* displaying strings, currencies, numbers, dates formats, weights and measures, etc. in a format an end-user will understand.
- *Data input:* the system must understand the format of the data being submitted to the application, this knowledge is essential for data integrity; for example, a French-speaking user will submit the date "9/11/04" for November 9th, 2004 (dd/mm/year), while an American English-speaking user will submit that date as 11/09/04 (mm/dd/year).

The internationalization of data-intensive Web applications can be achieved by replicating the system. This means that it is necessary to replicate the database, translate the HTML code embedded in the JSP files, and modify the Java code to manipulate the newly introduced set of encodings. The required translation language is assumed to be alphabet-based, and languages, which use symbols are not, considered (e.g., Chinese, Arabic, etc.).

5.3.1. Database extension and query rewriting

The main characteristic of a data-intensive Web application is the interaction with database servers, which dynamically provide the data to be published in the requested pages. The internationalization process requires modification of the information stored in the database, thus an extension of the database schema is necessary in order to support such translation.

- For each table T, representing an entity type with attributes subject to translation, a new table T_{trans} has to be created with $T_{\text{trans}} = \{\text{PK}_T, \text{language}, \{A_{t_1}, \ldots, A_{t_n}\}\}$.
- For each table R, representing a relationship type with attributes subject to translation, a new table $R_{\text{trans}}(\text{PK}_R, \text{language}, A_{t_1}, \ldots, A_{t_n})$ has to be created.(PK_T is the set of attributes representing the primary key, and $\{A_{t_1}, \ldots, A_{t_n}\}$ is the set of new attributes).

Rewriting the queries is indispensable in order to include the modified attributes in the result of each query. The characteristics of this rewriting process are as follows:

- The system will be used by English speaking users. This language is obtained by applying a custom rule to the preference language list coming from the HTTP request. French is denoted as the master language and English as the requested language.
- Each attribute A subject to translation is replaced with two attributes French-A and English-A containing its value in French and English, respectively.

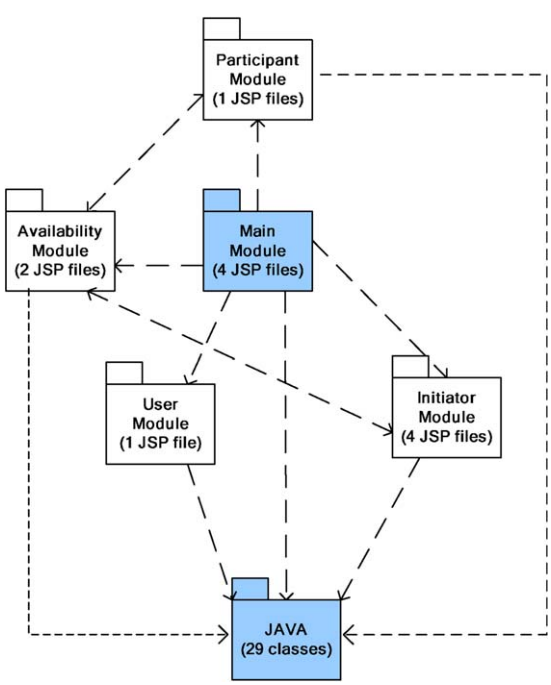

Fig. 6. Retrieving forgotten passwords (Team A).

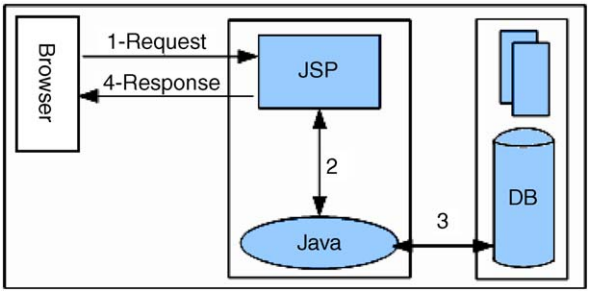

Fig. 7. International use (Team B).

Extending the database and rewriting the queries would solve the data display, the data representation and the data input issues. Only an additional table for each entity or a relationship-type object to be translated is added. In the worst-case situation, according to Aykin (1999), given a database containing m tables with attributes subject to translation, the number of new tables that have to be added is equal to 2m. Thus, all the database tables should be duplicated, and the queries nested in the Java component should be reworked for both systems (Fig. 7).

5.3.2. Translating the Web pages

The object of rewriting the queries is to extract from the database the requested data together with their translations in the desired language. All JSP files are thoroughly examined and all keywords are translated. Considering that English is the requested language, all attributes should be available in that language. The required modification does have a major impact on the architecture of Replan. A different implementation effort would be necessary to incorporate the *supporting international use* scenario in both teams, since their JSP, Java and database components are different and the development effort is reasonably high (Table 2).

5.4. Modifying the interface

Usability is no longer a luxury in Web development. As the popularity and acceptance of Web-based systems increase, the issues related to usability and UI design are increasing in frequency. While people may grow comfortable and reassured with respect to one of the most obvious issues of concern, security, they may find it difficult to feel reassured about some Web applications because of their lack of standards and practices associated with usability. By applying Nielsen's usability heuristics (1994, 2002), the daunting task of improving features in a data-intensive Web application can be addressed. Using this set of heuristics as a touchstone in the evaluation process, it becomes clear that the system suffers from usability issues such as the absence of a title or a header describing screen content, and an inconsistent icon design scheme and stylistic treatment across the system. The design of the JSP files that contain the HTML fragments making up the UI should minimize the user's memory load by keeping the same navigation bar and page layout. A flexible, minimalist aesthetic design is easier on the users' eyes and allows them to concentrate on the task at hand without distraction in the interface design (odd colors, for example). The use of a linear progression facilitates a narrative flow, as well as giving the user a feeling of control over the process. Matching the system and real-world expectations means that, when the task is "completing a form", the flow in completing that form should be a process as smooth and problem-free as possible.

Since this scenario is solely concerned with the presentation of information, separating the UI from the remainder of the application is sufficient to resolve all the violated heuristics. More specifically, this scenario can be satisfied by looking at the JSP files that suffer from the usability issues revealed through the application of Nielsen's usability heuristics, and then reworking those files. That being said, the modifications that will be brought to the UI can be implemented on an existing system at a late stage of development. Even though this scenario requires the modification of only one component and has no impact on the architecture, we decided to classify it as minor. UI design expertise is often nonexistent during development, and the time-consuming aesthetic modifications that the development team implement in the end could have been reduced, or even avoided, if the user had been involved in its design at the outset (Seffah et al., 2005). The objective here was not to list all the violated heuristics, but merely to state that they can be corrected without affecting the architecture of either application too severely.

5.5. Providing good help

Users do not tend to read printed documents before using a Web application, so online help needs to be easy to use and allow quick location of the necessary information. A "help system" should be well organized and should present users with a clear path to the information they need, either through context-sensitive help or by providing intuitive and advanced search capabilities. Since most Web applications are simply several pages linked together, if the user has a question or requires help, a simple hyperlink to the FAQ section usually does the trick.

There are many different types of Web application help systems. By far, the most popular method is simply to create one page per help item for every page on the website. Basically, the information required is all on one page. Users click on the hyperlink and obtain access to everything they could ever want to know about the site. A problem arises when they simply have a quick question to which the help option should generate a quick and simple answer, and yet they enter a world where they must scroll through massive amounts of data in the hope of finding the answer to the now forgotten question.

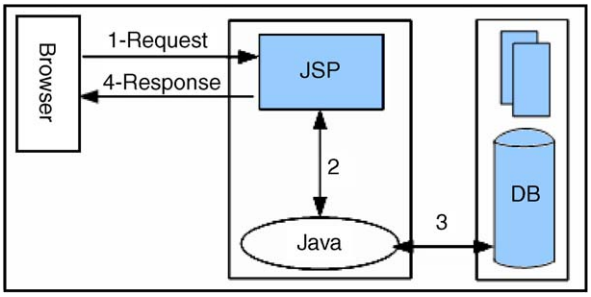

Fig. 8. Help system (Team B).

The other, more advanced method is a full-blown help application. Once the help icon is clicked, the user dives into an application for the help system. This type of application is more in common with online books. It is more thorough because it covers the entire application, including other aspects such as customization or the installation of client components.

A solution, which lies between the two in functionality, is proposed. This is an easy design solution where each page on the site has its own separate help page accessed by calling up the *Help.JSP* file, and which is going to be added to the JSP component. Every Web page would include this simple hyperlink (Fig. 8). The newly introduced page will, once accessed, extract the page name that called the help system and store it in a variable. That variable will then look up the corresponding help file in HTML format in a newly created table in the database that contains an index on the calling page's name field. Once retrieved, the help text will be inserted into the JSP file and will be presented to the browser. A "back" button, containing the link to the referring page should also be added, to return the browser to the original page that initiated the entire help journey. The database will contain one row in a table for every page. Each row has all the required text to be displayed for the application. Also, if the help text changes, all that is required is an update to the table row for the specific page's entry. There are no pages to change, and it is a clean and simple way to integrate a help function into the system. This solution is adequate because the Tomcat Web server can support server variables (such as REFERER_URL), which will notify the target page of the name of the originating JSP document. The implementation of this scenario is dependent on the size of the JSP component, more precisely the number of JSP files. According to Table 2, Team A has 15 JSPs, while Team B has 24. The greater the number of interfaces (JSP files), the bigger is the help system, since every interface should have its own help content. The addition of a help system has a minor impact on the architecture of Replan. The effort needed to incorporate this scenario is entirely dependent on the actual implementation of the JSP component and the Database. This entails doing nothing to the Web application, since the help system is completely self-contained.

6. Overall evaluation

6.1. Summary

If the required development effort is low and is the same for both Teams A and B, then we conclude that the required modifications are architecturally insensitive, since the two teams have distinct architectures. The *retrieving forgotten passwords* scenario seems to be independent of the architecture, and the proposed solution would necessitate the same effort for both systems. The Java component will be modified to retrieve the user's password from the database, and a lost password page (added to the Main module) is created. Fortunately, these modifications will leave the architecture unchanged.

If the envisioned usability modifications require the reworking of a relatively small number of architectural components, then the changes will have a minor impact on the architecture. In this case, the required changes necessitate a fairly small development effort. The *modifying interfaces* scenario necessitates the listing of all the interfaces that are lacking important usability guidelines established by Nielsen, and correcting them. In fact, if UI design expertise was present during architectural design, Nielsen's guidelines would have been properly followed and the effort required to go through all the interfaces could have been avoided. Also, for the *providing good help* scenario, a new table containing the help information has to be created. This requires only the modification of the database, and not the modification of major components. All JSP pages should be redesigned to include a link to the new Help page. The greater the number of JSP files, the larger the new Help database table. Therefore, both of these scenarios entail a minor impact on the architecture, given that it requires roughly the same development effort for both teams and does not involve the revision of major components.

In order to implement the *checking for correctness* scenario, a validation bean responsible for performing secure server-side validation on the data entered in the form is added to the Java component. For example, if the validation of the user's personal information form is required, then a validation bean with its respective validation rules is added to the User class. According to the J2EE development guidelines, business logic should be encapsulated in the Java component while the presentation would reside in the JSP component. In this implementation of the Web architecture, the JSP component contains some embedded Java code. To implement the validation bean, all inserted Java code in the JSP should first be moved to the Java. The development effort is rather significant and the architectural impact of the *checking for correctness* scenario is classified as intermediate.

The *supporting international use* scenario primarily involves modification of the database. Every table in the database is subject to replication, since data input and data display in an appropriate language should be ensured. The

greater the number of tables, the greater the effort to validate that scenario. Team A has five tables, while Team B has 11. Thus, the effort required to develop the database is considerably greater for the second team. Unfortunately, extending the database does not mean that the scenario is satisfied, as the JSP and Java components need to be revised as well. All interfaces should be translated. Generating a large number of JSP files would require a significant effort to translate the HTML fragments embedded in those files. Also, the database queries need to be rewritten to manipulate the newly introduced parameters in the database. In conclusion, the *supporting international use* scenario has a major impact on the architecture of Replan, since its implementation requires a substantial effort in terms of integrating it into two noticeably different architectures, and all the components of the architecture are modified. Table 3 lists the chosen scenarios and their impact on the Web architecture.

6.2. Discussion

The *retrieving forgotten password* scenario, which covered only one indicator of the effectiveness aspect of usability (accommodating mistakes) (Table 1), has no impact on the architecture. The *providing good help* scenario, which covers both indicators of the efficiency aspect of usability (supporting problem-solving and learnability), and the *modifying interfaces* scenario, which covers accelerating error-free portions and accommodating the mistake aspects of effectiveness, necessitate minor modifications to the architecture. The *checking for correctness* scenario covers four usability indicators, encompassing both the effectiveness and the efficiency aspects, has an intermediate impact on the architecture. The *internationalization* scenario supports all three aspects (effectiveness, efficiency and satisfaction) and has a major impact on the architecture. In addition, this scenario is the only one that supports

Table 3
Results of scenario evaluation

Usability scenario	Architectural impact	Files affected	Components modified
Checking for correctness	Intermediate	• 4 JSPs that contain forms • 3 Java classes	JSP and Java
Retrieving forgotten passwords	No	• 1 JSP file (Help.jsp) • 1 Java class (User.java)	JSP and Java
Providing good help	Minor	• All JSP files • Database tables	JSP and Database
Modifying interfaces	Minor	• All JSP files suffering from usability issues	JSP
Supporting international use	Major	• All JSP files • All Java classes containing SQL queries • Database tables	JSP, Java and Database

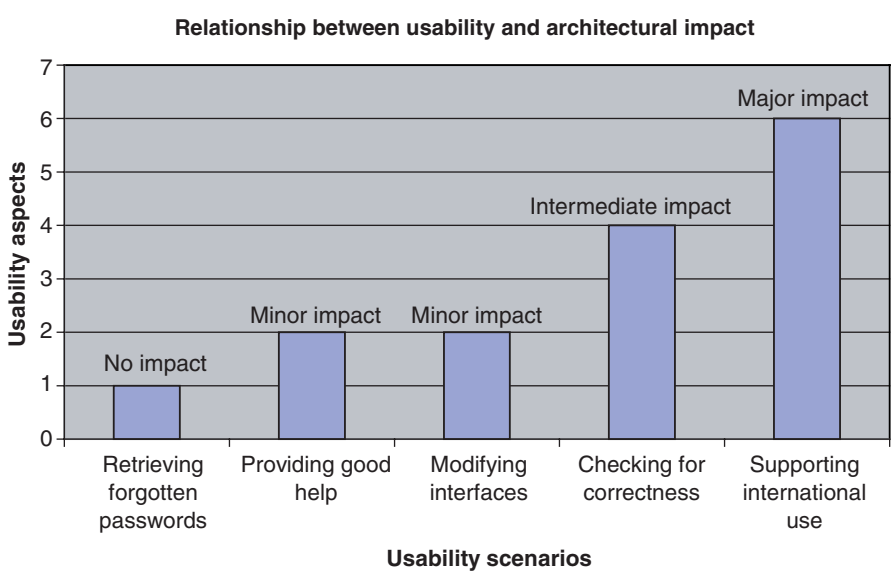

Fig. 9. Relationship between usability and architectural impact.

increasing the user's confidence and comfort. Hence, as shown in Fig. 9, there is a relationship, between the number of usability aspects of a scenario and the impact it will have on the architecture. The greater the number of usability aspects, the greater the impact of the scenario on the architecture. This work can be useful to a design team in evaluating and designing a Web architecture. It would enable them to select and implement the scenarios that can easily be incorporated into the architecture (the ones that cover a low number of usability aspects) at later stages of development. It would also allow them to consider, during the elaboration of the architecture, scenarios that require the overall structure of the system to be reworked. Those scenarios are said to be "complex", since they cover many cognitive aspects, such as increasing individual effectiveness, reducing the impact of mistakes and increasing the comfort level of the user.

Also, most of the chosen usability scenarios can be integrated into both existing systems without incurring major changes to the architectures, except for one, the *supporting international use* scenario. This is the only scenario that requires all Web components, as well as the database, to be reworked. It does not entail the same implementation effort for both systems. Having a greater number of Database tables means that the effort for updating and maintenance is higher, since every instance in that database has to be maintained. Therefore, the *supporting international use* has an impact on another quality attribute, maintainability. Regrettably, this last scenario is not the only one that has an impact on maintainability. If the separation of concerns is not established, as it should be, then the *checking for correctness* scenario also has an impact on that quality attribute. Problems can arise when applications contain a mixture of data access code, business logic and presentation code. Such applications are difficult to maintain because the interdependencies between all the components can cause strong ripple effects whenever a change is made at any given location. The Web components are difficult to reuse, as they depend on many other classes and require maintenance in multiple places.

7. Recommendations

Recent studies confirm that a significant part of the maintenance costs of software systems is spent on usability issues (Li and Henry, 1993). These high costs are largely due to the usability requirements, which will not be revealed until the software has been implemented or deployed. This analysis shows that the usability scenarios that entail a substantial architectural impact (checking for correctness and supporting international use) have an effect on the maintainability of a Web application. The system architecture was not designed to cope with the incorporation of late usability changes. We firmly believe that the architectural impact of those chosen usability scenarios could have been less serious if the architecture had been different. We discuss, in the following section, how the scenarios that entail a significant impact on the architecture, namely *checking for correctness* and *supporting international use*, can be instigated and their architectural impact minimized.

In any Web-based application, the data can be validated in two ways. The first method is the approach that was discussed previously, validation on the server side, while the second is to validate the data on the client side, before any of it is submitted to the server. This is usually achieved with JavaScript (JS) running on the client's browser. A few JS functions are created which handle common validation tasks. Activated by clicking the form's "submit" button, they test field content for proper formatting. If they detect a problem, the submission is aborted and an error message is displayed. Client-side validation has numerous advantages for both server and user. It lightens the load on the server and allows the data to be validated before it is sent for processing. Since validation takes place on the client's machine, there is no delay incurred by the client contacting a remote server. Besides the user getting a quicker response, server-processing power is conserved. The implementation of this approach in the architecture would require transferring the control responsible for validation to an external component responsible for regrouping all JavaScript validation files. Fig. 10 illustrates the proposed new architecture.

The main component that needs to be reworked in order to satisfy the *supporting international use* scenario is the database, as it is closely coupled to the database queries in the Java component. If this scenario was introduced during the architectural design phase, would the design of the database have been different? The software architecture is the artifact that embodies the earliest design decisions, and, if this scenario had been introduced during the architectural description phase, then we definitely believe that the system would have been designed in a way that would accommodate that scenario. Both teams would eventually have spent more time on the design of their database, since this has an effect on their development effort of the Java component. This scenario should have been related to decisions taken early in software development process. Hence, since architectural decisions can preclude the delivery of a usable system, we strongly believe that the inclusion of

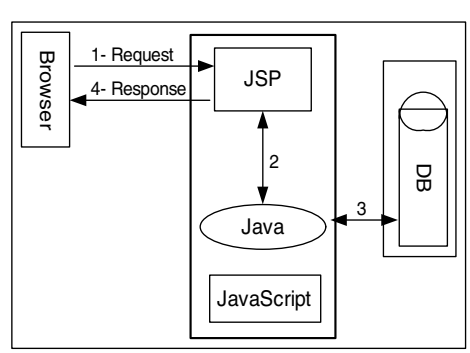

Fig. 10. Proposed architecture.

architecturally sensitive usability scenarios in an architecture-centric software development process such as UPEDU could help us design more usable systems.

8. Conclusion and future work

In the software and usability engineering communities, little work has been presented on how to integrate usability in the design of software architectures. We contribute to this ongoing research by discussing how SAAM, a scenario-based architecture analysis technique, can be adapted to assist software designers in designing software architectures, which support usability. Our application of SAAM consists of four major steps: (1) determining the set of usability scenarios, (2) describing the software architecture, (3) scenario evaluation, and (4) overall evaluation and discussion.

The first step of the method consisted in extracting from the work of Bass et al. the scenarios that are applicable to Web applications, along with their respective usability benefits. Potential Web-based design solutions were then proposed to highlight the impact of those scenarios on the Web architecture, once instigated. The proposed solutions were obtained by means of an inductive process, which guarantees that these solutions, albeit not necessarily the only ones, are possible. After analyzing two distinct architectures of the same system, our study shows that four of the six chosen scenarios could be implemented in an existing system without incurring major architectural changes. The usability scenario that covered one usability benefit (namely retrieving forgotten passwords) has no impact on the architecture, while the ones that covered two benefits (such as the modifying interfaces and providing good help) have a minor impact. In fact, the scenarios that revealed a large number of usability aspects (checking for correctness and supporting international use) have a major impact on the architecture. There is, therefore, a relationship between usability scenarios and their architectural impact. Moreover, besides having a serious impact on the architecture, these previous scenarios seem to have an effect on the maintainability of the system if they are implemented at later stages of development. This effect could be avoided if the usability requirements are defined and considered in the architectural description phase. Both teams constructed their architectures without paying sufficient attention to usability requirements and whether or not they can easily accommodate late modifications. This is why relating system usability to architectural decisions taken during the design phase is crucial, and the architecture should be designed to incorporate usability requirements. Software development processes, which have evolved from human-centered design or human–computer interaction, like ISO standard 13407, do not define software architectures, and therefore the usability scenarios that have an impact on the architecture cannot really be implemented. By contrast, since UPEDU® is defined as architecture-centric, the next step of this research will be to explore how it can accommodate user concerns and also to investigate how the architecturally sensitive usability aspects can be integrated into the process.

Acknowledgements

We are grateful to the students who participated in the project course during the winter 2002 semester, and we thank them for their collaboration in providing the details of their work.

This work was supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) under grants A0141 and 121923-03.

References

Aykin, N., 1999. Internationalization and localization of web sites. In: Proc. of the 8th International Conference on Human–Computer Interaction (HCI), Munich, Germany, pp. 1218–1222.

Bass, L., John, B.E., 2003. Linking usability to software architecture patterns through general scenarios. Journal of Systems and Software 66, 187–197.

Bass, L., John, B.E., Kates, J., 2001. Achieving Usability Through Software Architecture, CMU/SEI-TR-2001-005. Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA.

Bengtsson, P.O., Bosh, J., 2002. Assessing optimal software architecture maintainability. In: Proc. of the 5th European Conference on Software Maintainability and Reengineering.

Bevan, N., 1995. Measuring usability as quality of use. Journal of Software Quality (4), 115–140.

Clements, P., Kazman, R., Klein, R., 2001. Evaluating Software Architectures: Methods and Case Studies. Addison Wesley, MA.

Constantine, L., Lockwood, L., 1999. Software for Use: A Practical Guide to the Models and Methods of Usage-Centered Design. Addison-Wesley, New York.

Dobrica, L., Niemela, E., 2002. A Survey on Software Architecture Analysis Method. IEEE Transactions on Software Engineering 28 (7).

Folmer, E., Bosch, J., 2002. Architecting for usability: a survey. Journal of Systems and Software, 61–78.

Haggander, D., Bengtsson, P.O., Bosh, J., Lundberg, L., 1999. Maintainability myth cases: performance problems in parallel applications. In: Proc. of the IASTED 3rd International Conference on Software Engineering and Applications, pp. 288–294.

Hix, D., Hartson, H.R., 1993. Developing User Interfaces: Ensuring Usability through Product and Process. John Wiley and Sons.

Inderjeet, I., Stearns, B., Johnson, M., 2002. Designing Enterprise Applications with the J2EE Platform. Addison-Wesley, CO.

ISO 13407, 1999. Human-centered design process for interactive system teams.

ISO/IEC, 1998. 9241-11 Ergonomic requirements for office work with visual display terminals (VDTs)—Part 11: Guidance on usability, ISO/IEC 9241-11:1998.

Kazman, R., Abowd, G., Bass, L., Webb, M., 1994. SAAM: a method for analyzing the properties of software architectures. In: Proc. of the 16th International Conference on Software Engineering, Sorrento, Italy, pp. 81–90.

Kazman, R., Abowd, G., Bass, L., Clements, P., 1996. Scenario-based analysis of software architecture. IEEE Software 13 (6), 47–56.

Landauer, T.K., 1995. The Trouble with Computers: Usefulness, Usability and Productivity. MIT Press, Cambridge.

Li, W., Henry, S., 1993. OO metrics that predict maintainability. Journal of Systems and Software, 111–122.

Nielsen, J., 1993. Usability Engineering. Academic Press Inc., Boston.

Nielsen, J., 1994. Heuristic evaluation. In: Nielsen, J., Mack, R.L. (Eds.), Usability Inspection Methods. John Wiley & Sons, New York.

Nielsen, J., 2000. Designing Web Usability: The Practice of Simplicity. New Riders Publishing, Indianapolis, IN.

Nielsen, J., 2002. Alertbox, December 2002. Available from: <http://www.useit.com/alertbox/20021223.html>.

Rafla, T., Oketokoun, R., Wiklik, A., Desmarais, M., Robillard, P.N., 2004. Accommodating usability driven changes in existing software architecture. In: Proc. of the IASTED 8th International Conference on Software Engineering and Applications (SEA), Cambridge, USA, pp. 150–154.

Robillard, P.N., Kruchten, P., D'astous, P., 2003. Software Engineering Process with UPEDU. Addison Wesley, Boston.

Schneiderman, B., 1998. Designing the User Interface, third ed. Addison-Wesley, Reading, MA.

Seffah, A., Gulliksen, J., Desmarais, M. (Eds.), 2005. Human-Centered Software Engineering: Integrating Usability in the Development Process. Springer Publishing.

Shackel, B., 1991. Usability—context, framework, design and evaluation. In: Shackel, B., Richardson, S. (Eds.), Human Factors for Informatics Usability. Cambridge University Press, pp. 21–38.

Tamer Rafla is a gradate student in the Software Engineering Research Lab at the Ecole Polytechnique de Montreal. He has held several positions in the telecommunications and software industries after graduating from Concordia University. His research interests include Human–Computer-Interaction, requirements engineering, software architecture and software processes. He is an academic member of the IEEE and the Quebec Order of Engineers, and a founding member of the WWISA.

Pierre N. Robillard is a full professor in the Department of Computer Engineering at the Ecole Polytechnique de Montreal where he heads the Software Engineering Research Lab. His research interests include cognitive aspects in software engineering, software collaborative work, software measurement, software engineering education and software processes. He is the author or co-author of more than 120 scientific papers or conferences and three books, the latest being "Software Engineering Process with the UPEDU", Addison Wesley. He is a licensed professional engineer in the province of Quebec and in Canada, and member of the IEEE, ACM, EACE and CIPS.

Michel Desmarais is an associate professor in the Computer Engineering Department of the Ecole Polytechnique de Montreal since 2002. His field of expertise is in the domains of Human–Computer Interactions and Artificial Intelligence. He has over 10 years of experience in software project management. He was principal researcher of the Human–Computer Interaction group at the Computer Research Institute of Montreal between 1990 and 1998, where he directed a research program in HCI and computer-assisted learning, and was involved in a number of research projects in close collaboration with private corporations. From 1998 to 2002, he directed the Web services department in a private company and directed a number of Web-based software R&D projects. He is the author of a book, 10 scientific journal publications and over 20 book chapters and conferences papers.